

Topic: Queue using Singly Linked List

Q. Describe the implementation of a Queue using a Singly Linked List. Detail the logic for the Enqueue and Dequeue operations.

[DEF] Definition to Remember

A Queue follows the FIFO (First In, First Out) principle. Implementing it using a Singly Linked List allows dynamic sizing and avoids the memory wastage or circular logic needed in array implementations. It uses two pointers: front (for deletion) and rear (for insertion).

Queue Implementation using Singly Linked List

To achieve O(1) time complexity for both Enqueue and Dequeue, we must maintain references to both ends of the list.

Queue Pointer	Linked List Equivalent	Used For
Front	`head` pointer	**Dequeue** (Delete from beginning)
Rear	`tail` pointer	**Enqueue** (Insert at end)

2. Enqueue Operation (Insert at End)

Adds a new element to the rear of the queue.

Algorithm:

1. All
2. new
3. If
4. Els

3. Dequeue Operation (Delete from Beginning)

Removes the element from the front of the queue.

Algorithm:

1. If
2. tem
3. val
4. fro
5. If
6. fre
7. ret

4. Advantages of Linked List Implementation

* Dynamic Sizing: Queue can grow infinitely (bounded only by heap memory). No "Queue Full" fixed limit.

* O(

[MUST-WRITE] Must-Write Points (for 10 marks)

- 1. Queue is FIFO. Linked List implementation allows dynamic sizing without array constraints.
- 2. Uses two pointers: front (points to first node) and rear (points to last node).
- 3. Enqueue = Insert at End (using rear pointer). Time complexity is O(1).
- 4. Dequeue = Delete from Beginning (using front pointer). Time complexity is O(1).
- 5. Enqueue Logic: rear->next = new_node; rear = new_node;

6. Dequeue Logic: temp = front; front = front->next; free(temp);

7. If front becomes NULL after a dequeue, rear must also be set to NULL.

[RECALL] Quick Recall

Queue via Linked List -> Dynamic Size -> front (Head) for Dequeue ($O(1)$) -> rear (Tail) for Enqueue ($O(1)$) -> If empty: front = rear = new_node